

Yggdrasil ERP — Security Architecture

Mimir Labs Technical Publication

Document Version: 1.0

Date: March 2026

Classification: Public

Executive Summary

Security in Yggdrasil ERP is treated as an architectural property rather than an operational afterthought. The platform integrates identity management, authorization controls, tenant isolation, encryption, audit logging, and infrastructure hardening into a coherent security model designed for multi-tenant enterprise environments.

The goal of the security architecture is not simply to prevent unauthorized access. It is to ensure that enterprise operations remain trustworthy, traceable, and isolated even when the system is operating at scale across multiple organizations.

This paper explains how Yggdrasil's security controls operate across the authentication layer, authorization model, database isolation boundaries, infrastructure deployment model, and operational governance framework.

1. Security Philosophy

The security posture of Yggdrasil is built on five guiding principles.

First, **least privilege** governs all access decisions. Users and services receive only the capabilities required to perform their operational role.

Second, **defense in depth** ensures that no single failure can compromise the system. Controls exist simultaneously at the application, database, infrastructure, and operational levels.

Third, **tenant isolation** is treated as a fundamental invariant. Data belonging to one organization must remain inaccessible to all others, regardless of application logic or integration behavior.

Fourth, **fail-secure behavior** ensures that ambiguous situations default to denial rather than permissive access.

Fifth, **auditability** guarantees that meaningful system activity can be reconstructed from recorded events. Security-relevant actions must always leave an observable trail.

These principles shape the design of every layer in the system rather than appearing as optional operational controls.

2. Identity and Authentication

Identity verification establishes the foundation of the Yggdrasil security model. Passwords are protected using modern key-derivation algorithms designed to resist offline brute-force attacks. Each password hash is generated with a unique random salt and a high iteration count to increase computational cost for attackers.

Multi-factor authentication is supported using time-based one-time passwords. During enrollment, a user registers an authenticator application which generates rotating verification codes tied to a shared secret. Recovery codes provide a controlled fallback mechanism if the authenticator device becomes unavailable.

To protect against credential-guessing attacks, the system implements progressive account lockout. Repeated failed login attempts trigger increasingly longer lockout periods, preventing automated brute-force activity while preserving legitimate user access.

Session management uses short-lived access tokens and longer-lived refresh tokens delivered through secure HTTP-only cookies. This approach minimizes exposure to browser-side attacks while maintaining a responsive user experience.

3. Authorization Model

Once identity has been established, the authorization layer determines what actions a user may perform.

Yggdrasil uses a role-based access control system organized around a hierarchy of operational roles. These roles range from system administrators with cross-tenant authority to read-only users who may inspect data but not modify it.

Authorization decisions are enforced centrally through middleware that intercepts every API request. The middleware evaluates the user's role, the module being accessed, and the operation being attempted before allowing execution to proceed.

This centralized enforcement model ensures that permissions remain consistent across the system. Newly introduced endpoints automatically inherit the same security checks rather than requiring developers to manually implement authorization logic for each route.

Administrative functions such as configuration management, data restoration, and user provisioning are protected by additional privilege checks beyond standard module permissions.

4. Multi-Tenant Isolation

Yggdrasil is designed for multi-tenant operation, where multiple organizations share a single application environment while maintaining strict data isolation.

Each tenant's data is associated with a unique tenant identifier embedded within operational records. Application logic automatically scopes database queries to the active tenant context derived from the authenticated session.

To prevent cross-tenant exposure caused by programming errors, isolation is also enforced at the database level. PostgreSQL row-level security policies ensure that queries cannot return records belonging to other tenants, even if application code fails to apply appropriate filtering.

This layered isolation model significantly reduces the risk that a single defect could compromise organizational data boundaries.

Real-time event channels and integration mechanisms follow the same isolation rules. Events are scoped to tenant-specific streams so that activity in one organization cannot be observed by another.

5. Encryption and Data Protection

Protection of sensitive data relies on both transport-level and storage-level encryption.

All external communication with the platform occurs over encrypted TLS connections. WebSocket communication uses secure transport protocols to maintain the same guarantees for real-time data streams.

Sensitive configuration values and integration credentials are encrypted using modern symmetric encryption algorithms before being stored. Passwords are never stored in reversible form and cannot be reconstructed from their stored hashes.

Database storage volumes and backup archives are encrypted at the filesystem or archive level, ensuring that data remains protected even if storage media are accessed outside the application environment.

Cryptographic algorithms and key lengths follow modern industry recommendations, and weak or deprecated algorithms are explicitly prohibited within the platform.

6. Input Validation and Application Protection

Application-layer protections prevent malicious input from reaching critical system components.

All database interactions use parameterized queries rather than dynamically constructed SQL statements. This eliminates the primary attack vector for SQL injection.

The API layer validates incoming request payloads against strict schemas. Unexpected fields and malformed values are rejected at the boundary before business logic executes.

Additional safeguards validate identifiers, sanitize file paths used by attachment systems, and ensure that output serialization prevents cross-site scripting vulnerabilities.

These measures combine to create a defensive boundary between external clients and internal system components.

7. Audit and Security Logging

Operational transparency is a central element of the Yggdrasil security architecture. Every meaningful data mutation is recorded in a centralized audit log that captures the acting user, the affected entity, and the before-and-after values of modified data.

Lifecycle transitions executed by the state machine service are recorded explicitly so that the operational history of critical objects can be reconstructed at any time.

In addition to operational audits, the system records security events such as authentication attempts, authorization denials, administrative actions, and configuration changes. These records allow operators to investigate incidents, analyze suspicious behavior, and demonstrate compliance with governance requirements.

Log storage is protected through access controls and automated rotation policies to prevent tampering and uncontrolled growth.

8. Infrastructure Security

The deployment architecture minimizes external attack surface by limiting direct exposure of application services.

Production deployments use a secure edge gateway that terminates public traffic and forwards requests through encrypted tunnels to the internal application host. This design prevents direct internet access to application ports.

Host systems follow standard hardening practices including restricted SSH access, intrusion detection mechanisms, automated security updates, and execution under a dedicated non-root service account.

Containerized deployment environments isolate application components while restricting network communication to explicitly defined channels.

These infrastructure measures complement the application-level controls described earlier.

9. Security Governance and Compliance

Beyond technical controls, Yggdrasil operates within a structured security governance framework. Formal security policies define expectations for access control, encryption standards, data classification, incident response, change management, and vendor risk management.

Operational monitoring and vulnerability management processes ensure that newly discovered risks are identified and remediated within defined timelines.

Security incidents follow a documented response procedure that includes detection, containment, investigation, recovery, and post-incident review.

This governance structure ensures that security practices evolve alongside the platform as it grows in scope and adoption.

10. Secure Development Lifecycle

Security considerations are integrated into the development process itself.

The codebase follows secure coding practices designed to minimize memory safety issues and common programming vulnerabilities. Static analysis, automated testing, and dependency scanning run as part of the continuous integration pipeline.

Schema validation and migration verification ensure that database changes cannot introduce unintended side effects. Automated testing frameworks verify both application behavior and security-sensitive logic before code is merged.

This development discipline reduces the likelihood that vulnerabilities will enter the system during normal feature development.

Conclusion

The Yggdrasil security architecture is designed to protect both operational integrity and organizational data boundaries in a multi-tenant enterprise environment.

By combining centralized authentication, strict authorization controls, database-level tenant isolation, strong encryption practices, comprehensive audit logging, and hardened infrastructure, the platform provides a layered defense against both accidental exposure and deliberate attack.

These controls allow the system to maintain trustworthy operational records while supporting the broader goal of the Mimir Labs architecture: coherent and accountable enterprise systems built on governed data and observable operations.

Copyright 2026 Mimir Labs. All rights reserved.